

软件度量及应用

第十讲

面向对象度量——CK度量方法

主讲教师：杨 柳

yangliu@csu.edu.cn

软件度量方法

软件规模度量 Size

代码行 LoC

功能点 FP

特征点 FeP

用例点 UCP

子主题 5

软件复杂性度量 Complexity

控制流结构-圈复杂度

数据流结构-模块、扇入扇出

数据结构-程序中的数据结构

面向对象度量 Object-Oriented

CK度量方法

LK度量方法

MOOD方法

软件质量度量 Quality

功能性

可靠性

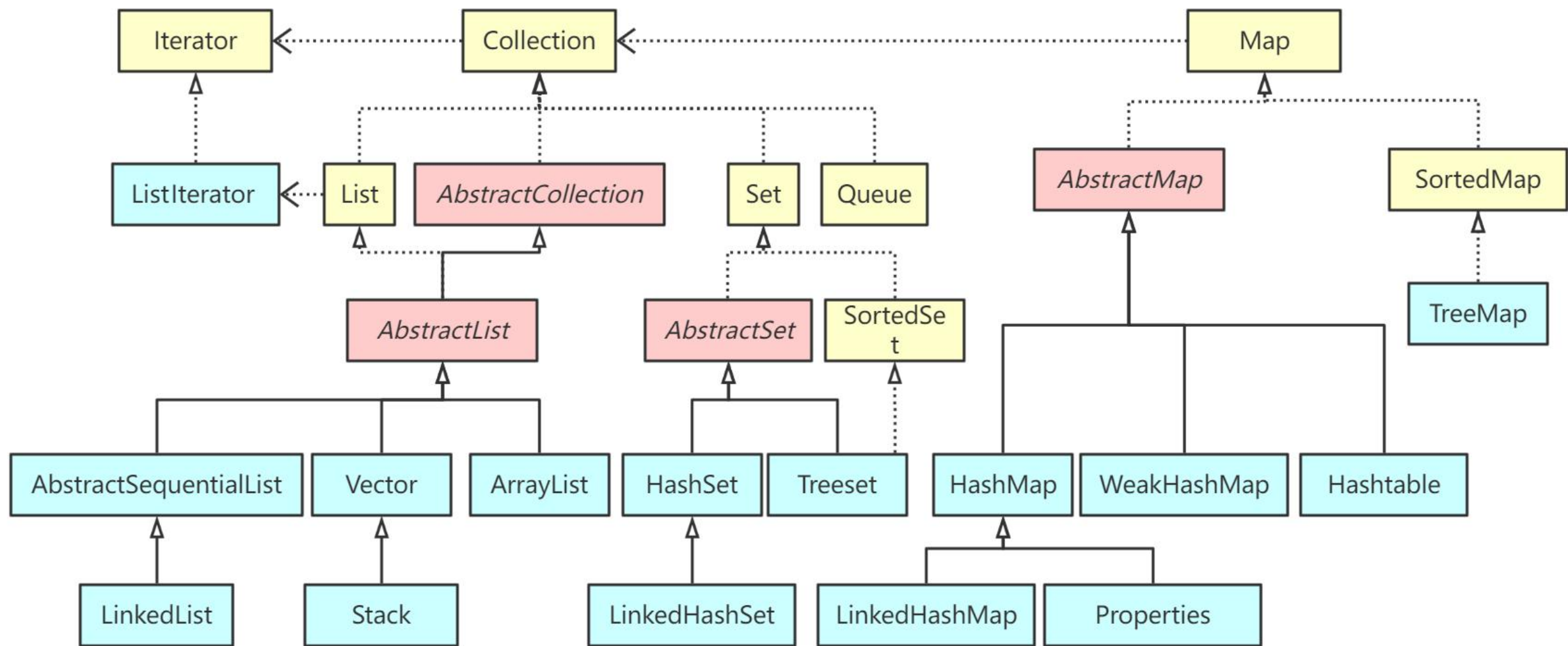
可用性

效率

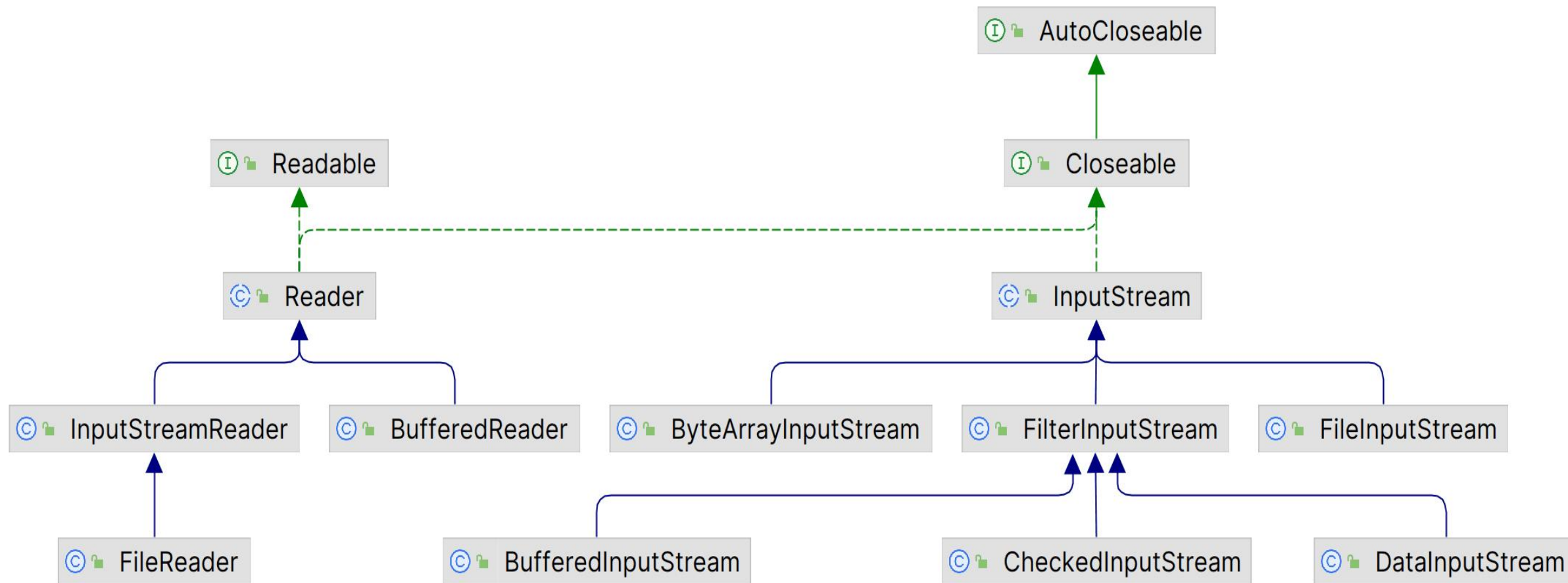
可维护性

可移植性

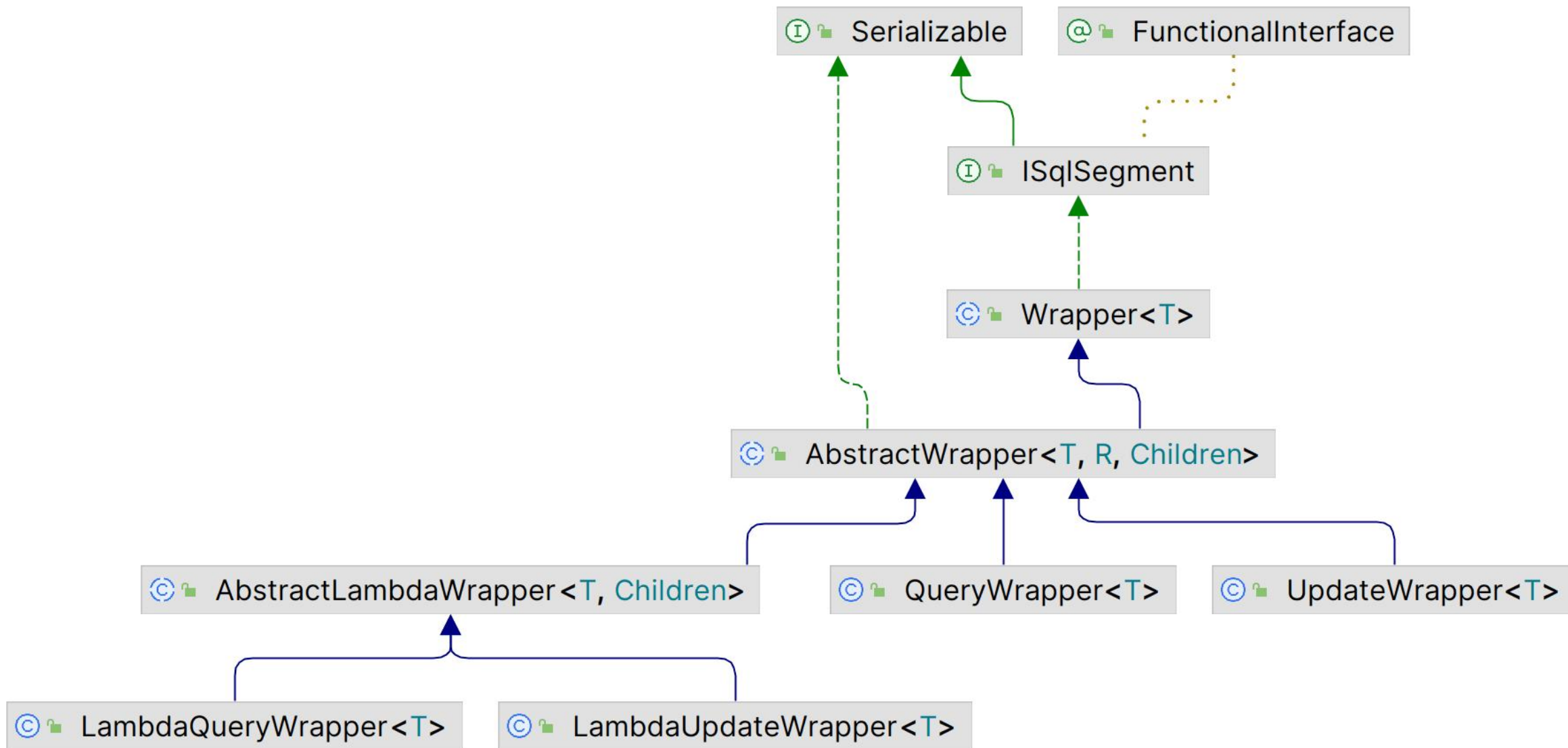
Java语言的集合Collection相关数据结构



Java语言的处理输入流的相关类和接口设计



开源对象关系映射(ORM)框架MyBatis-Plus的相关接口设计



CONTENTS

目录

1. 面向对象程序设计的特点
2. 面向对象度量的三类方法
3. 面向对象度量CK度量方法
4. CK度量方法的应用与实现

1. 面向对象程序设计的特点

熟悉面向对象软件程序设计的特点

1. 面向对象程序设计的特点

提问：如何度量面向对象软件？

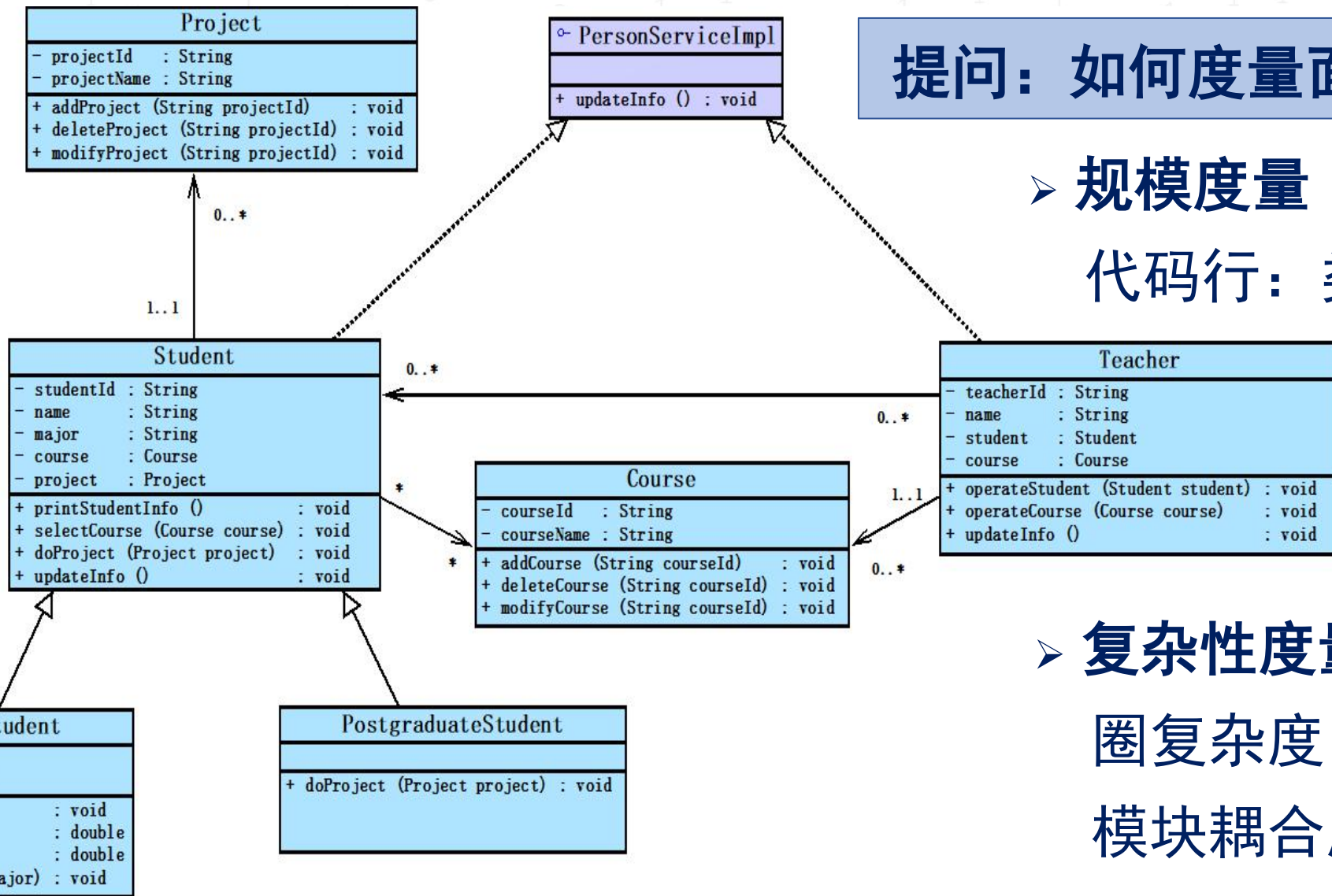
➤ 规模度量

代码行：类、方法

➤ 复杂性度量

圈复杂度：方法

模块耦合度：类

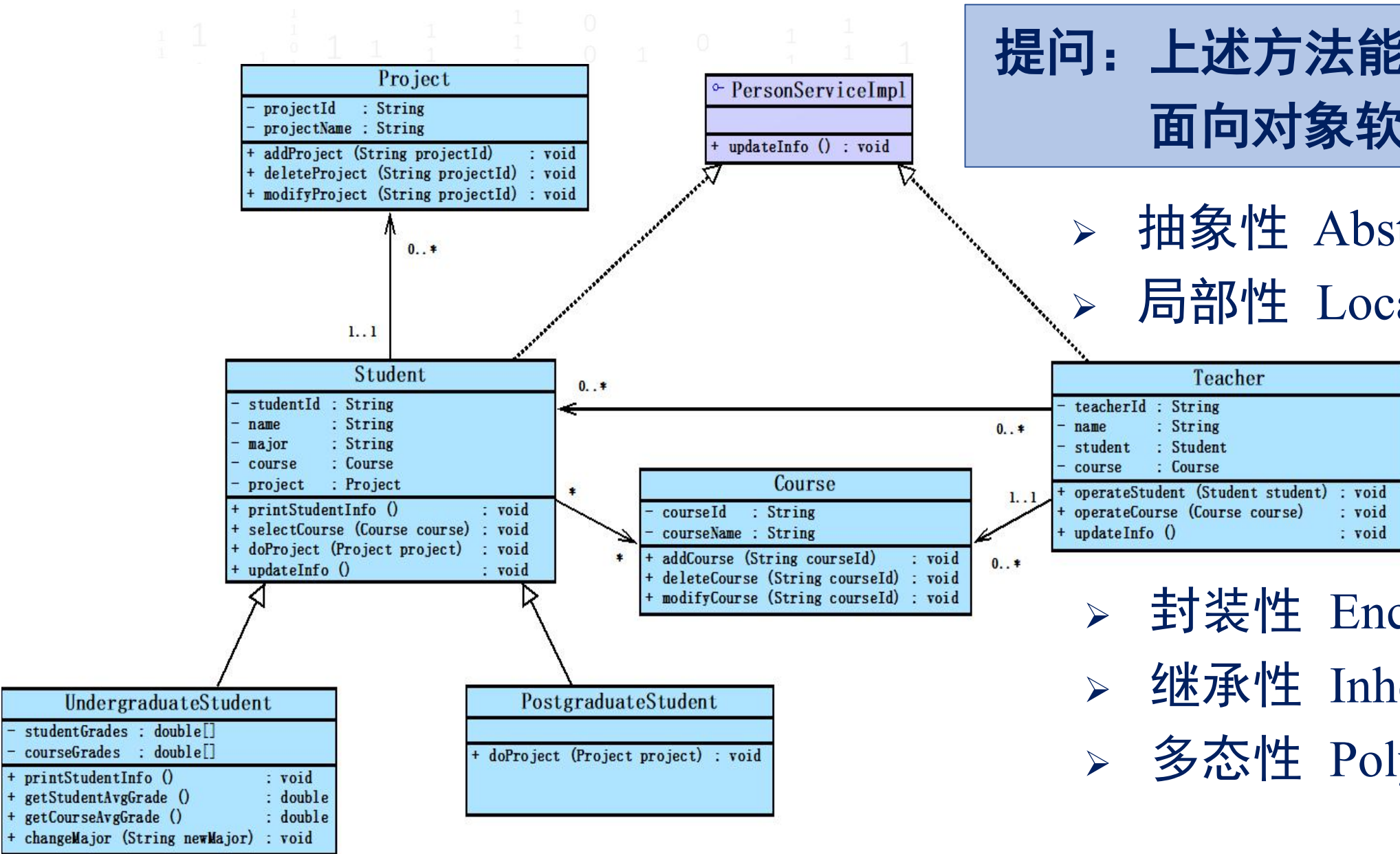


1. 面向对象程序设计的特点

提问：上述方法能够充分度量面向对象软件吗？

- 抽象性 Abstraction
- 局部性 Localization

- 封装性 Encapsulation
- 继承性 Inheritance
- 多态性 Polymorphism



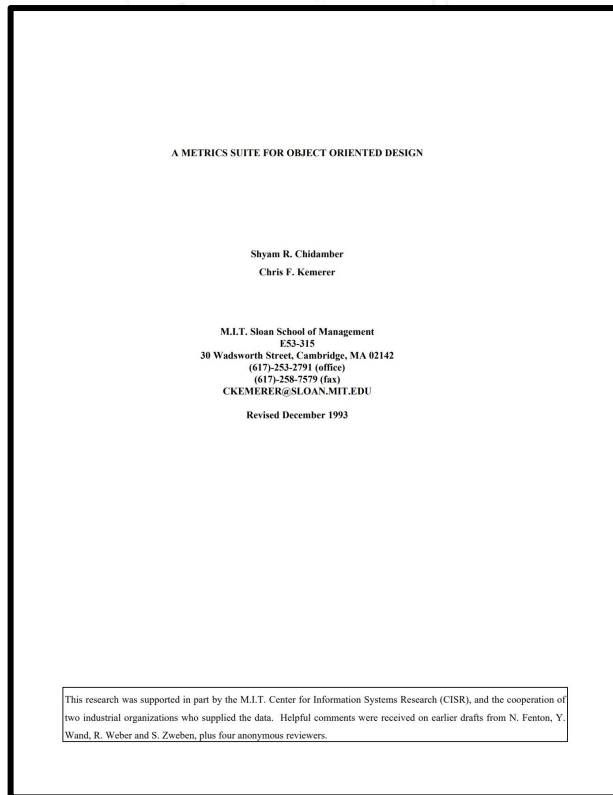
2. 面向对象度量的三类方法

熟练掌握面向对象软件度量方法

2. 面向对象度量的三类方法

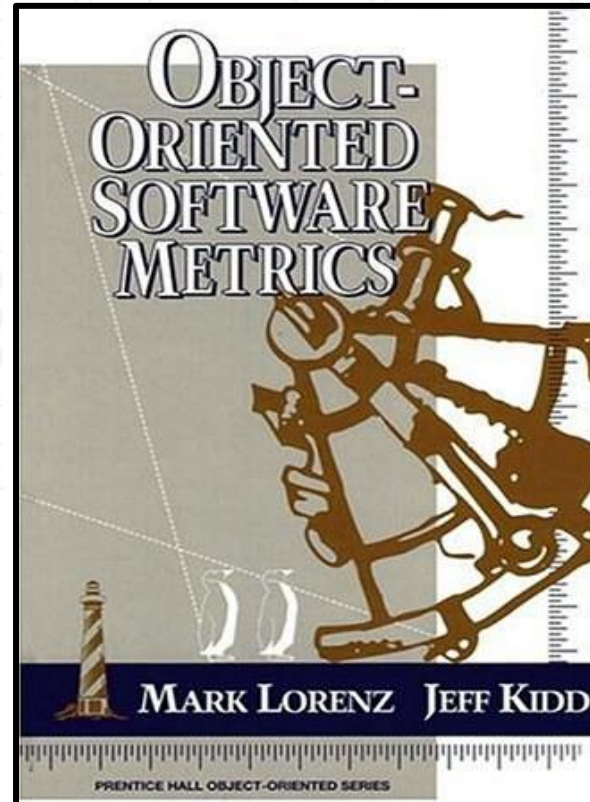
CK度量方法

(*S.R.Chidamber*
& *C.F.Kemerer*, 1994)



LK度量方法

(*M.Lorenz & J.Kidd*, 1994)



MOOD度量方法

(*A.Ran*, 1996)



3. 面向对象度量——CK 度量方法

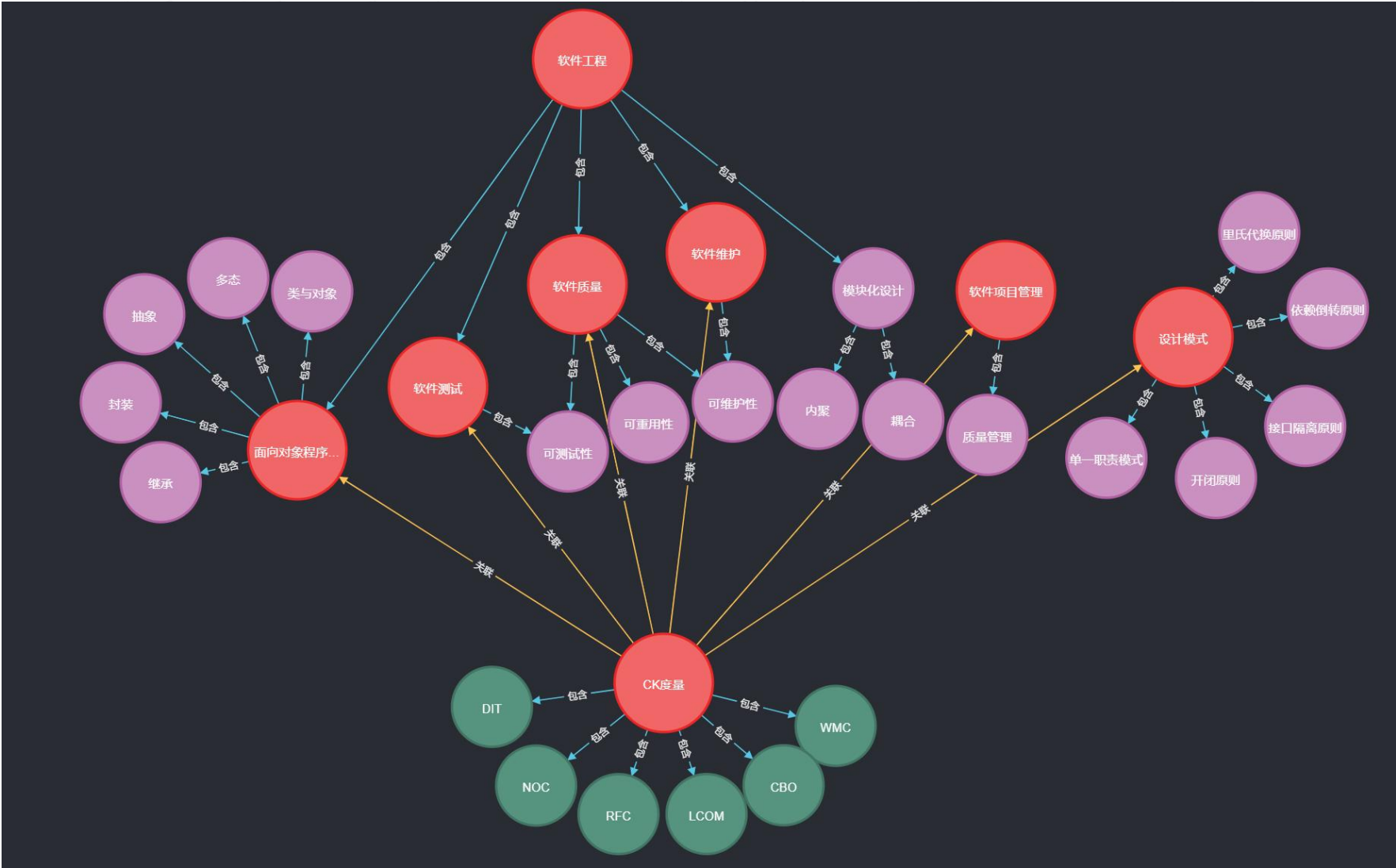
运用CK度量方法对面向对象的设计与代码进行改进

3. 面向对象度量——CK 度量方法

度量方法	度量点	度量对象	度量类型
传统方法	圈复杂度 (Cyclomatic Complexity, CC)	类方法	复杂性
	代码行 (Line of Code , LOC)	类/类方法	规模
CK度量方法	类加权方法数 (Weighted Methods per Class , WMC)	类/类方法	抽象性/规模/复杂性
	类响应数 (Response For a Class , RFC)	类/类方法	抽象性/规模/复杂性
	继承树深度 (Depth of Inheritance Tree , DIT)	类	继承性
	子类数 (Number of Child, NOC)	类	继承性
	内聚缺乏度 (Lack of COhesion in Methods , LCOM)	类/类方法	封装性/模块化
	对象耦合 (Coupling Between Objects , CBO)	类	封装性/模块化/复杂性

3. 面向对象度量——CK 度量方法

CK 度量方法相关知识图谱



3. 面向对象度量——CK 度量方法

(1) 类加权方法数 (Weighted Methods per Class, WMC)

■ 度量方法

定义：WMC是对类的规模与复杂性的度量

$$WMC = \sum_{i=1}^n C_i$$

其中， C_i 是类中方法 M_i 的复杂度， n 是一个类中方法数量

- 复杂性：方法 M_i 的复杂度可用其圈复杂度计算
- 规模：若所有方法的复杂度都看作一致，则 $WMC = n$

Student	
- studentId	: String
- name	: String
- major	: String
- course	: Course
- project	: Project
+ printStudentInfo ()	: void
+ selectCourse (Course course)	: void
+ doProject (Project project)	: void
+ updateInfo ()	: void

提问：一个类中的 set ()、get ()方法是否计入？

3. 面向对象度量——CK 度量方法

(1) 类加权方法数 (Weighted Methods per Class, WMC)

■ 度量实例

父类 Student:

$$\text{WMC}(\text{Student}) = 4$$

子类 Undergraduate、Postgraduate:

$$\text{WMC}(\text{Undergraduate}) = 8 ?$$

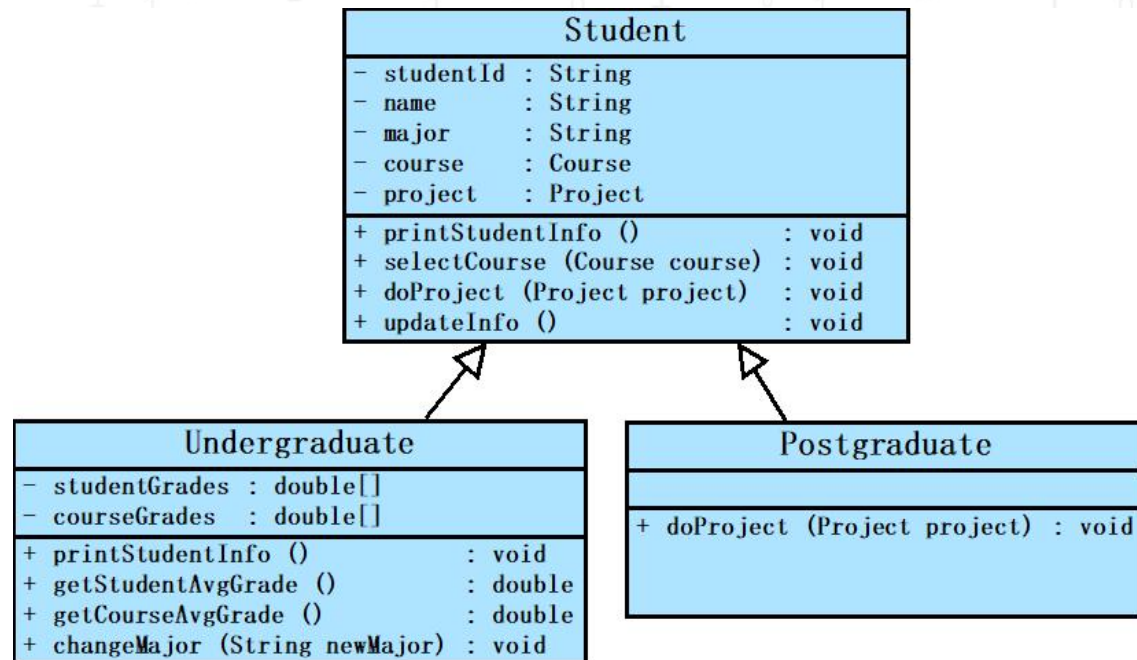


$$\text{WMC}(\text{Postgraduate}) = 5 ?$$

子类 Undergraduate、Postgraduate:

$$\text{WMC}(\text{Undergraduate}) = 4$$

$$\text{WMC}(\text{Postgraduate}) = 1$$



3. 面向对象度量——CK 度量方法

(1) 类加权方法数 (Weighted Methods per Class, WMC)

■ 应用意义

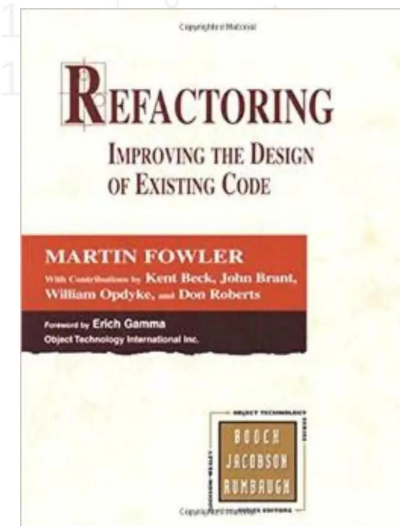
- WMC度量类规模与封装性，WMC值越大表明类的职责越多，对类的可读性、可扩展性、可维护性的影响就越大
- WMC值越大，开发和维护类所需时间与工作量越大
- WMC值越大，父类对子类的影响越大，越难以继承



代码坏味道：上帝类（God Class），违背“单一职责”原则

如何改进设计？

——将类分解为两个或多个类



3. 面向对象度量——CK 度量方法

(2) 类响应数 (Response of a Class, RFC)

■ 度量方法

定义：RFC是指对一个类对象的消息进行响应的方法数

$$RFC = |RS|$$

$$RS = \{M\} \cup_{\text{all } i} \{R_i\}$$

其中，RS 是一个类的响应集合

M 是一个类的方法集合

$\{R_i\}$ 是被方法 i 调用的方法集合

```
1 public class CourseServlet{
2     private Student Li_Bowen;      //学生实例
3     private String teacherName;    //教师名字
4
5
6     //查询已修的学分
7     public double queryObtainedCredits(){
8         return Li_Bowen.selectedCourseCredits();
9     }
10
11    //选择教师所授课程
12    public String selectCourse(string teacherName){
13        return Li_Bowen.getCourseArray(teacherName);
14    }
15 }
```

3. 面向对象度量——CK 度量方法

(2) 类响应数 (Response of a Class, RFC)

■ 度量实例

$RS(\text{CourseServlet})$
 $= \{ M \} \cup_{\text{all } i} \{ R_i \}$
 $= \{ \text{queryObtainedCredits}, \text{selectCourse} \}$
 $\cup \{ \text{selectedCourseCredits}, \text{getCourseArray} \}$

$RFC(\text{CourseServlet})$
 $= | RS(\text{CourseServlet}) |$
 $= 4$

```
1 public class CourseServlet{
2     private Student Li_Bowen;      //学生实例
3     private String teacherName;    //教师名字
4
5
6     //查询已修的学分
7     public double queryObtainedCredits(){
8         return Li_Bowen.selectedCourseCredits();
9     }
10
11    //选择教师所授课程
12    public String selectCourse(string teacherName){
13        return Li_Bowen.getCourseArray(teacherName);
14    }
15 }
```

3. 面向对象度量——CK 度量方法

(2) 类响应数 (Response of a Class, RFC)

■ 应用意义

- RFC 通过一个类的方法及方法与其他类的交互，度量类的复杂性与封装性
- RFC 值越大，类的设计实现越复杂，测试与调试就越难
- 在测试阶段，可以利用RFC 度量结果规划类的测试时间



代码坏味道：依恋情结（Feature Envy），可能违背“单一职责”原则

如何改进设计？

—— 将大量使用其他类成员的方法迁移到它所使用的那个类中

3. 面向对象度量——CK 度量方法

(3) 继承树深度 (Depth of Inheritance Tree , **DIT**)

■ 度量方法

定义：DIT 是指从当前类节点到继承树根的路径的最大深度

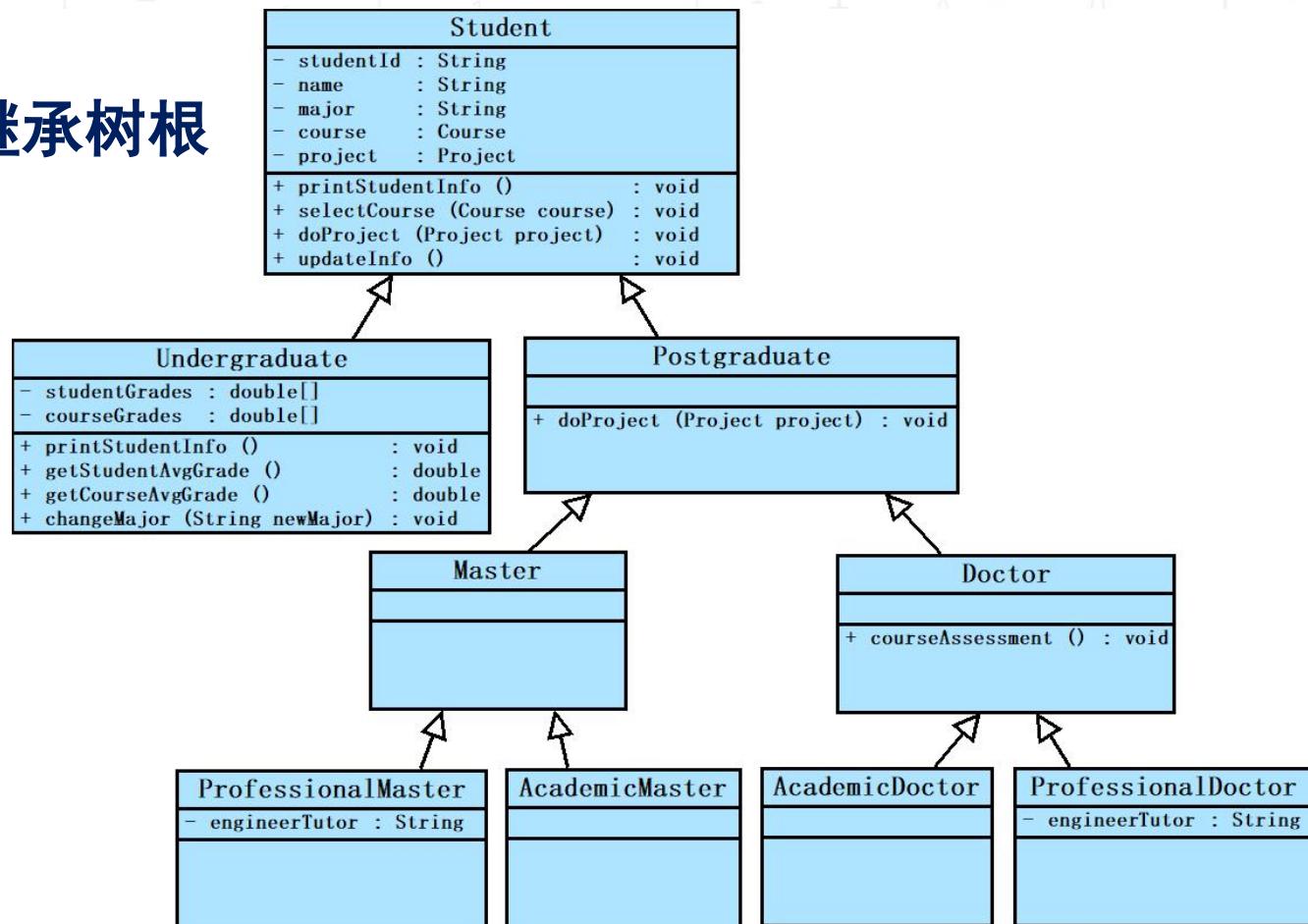
■ 度量实例

$DIT(\text{Student}) = 0$

$DIT(\text{Undergraduate}) = 1$

$DIT(\text{Master}) = 2$

$DIT(\text{AcademicMaster}) = 3$



3. 面向对象度量——CK 度量方法

(3) 继承树深度 (Depth of Inheritance Tree , **DIT**)

■ 应用意义

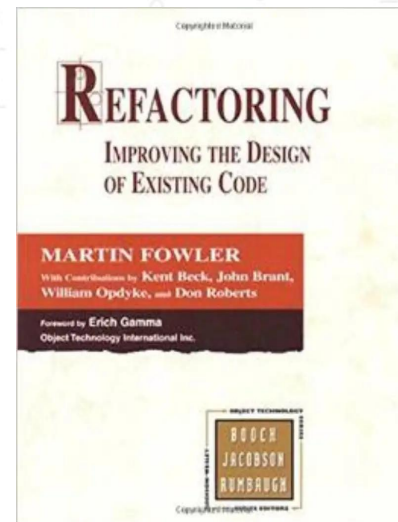
- 一个类的 DIT 值越大，其作为子类继承的方法数就越大，方法重用性就越高
- 越深的继承树涉及到更多的类和方法，表明类的设计实现复杂性越大
- 类的 DIT 值越大，越难预测类行为，类实现代码的可理解性与可维护性越差



代码坏味道：违背“里氏替换”原则

如何改进设计？

——取消子类与父类的继承关系，重新设计两者关系，
如子类独立门户



3. 面向对象度量——CK 度量方法

(4) 子类数 (Number of Children, **NOC**)

■ 度量方法

定义：NOC是指一个类的直接子类数

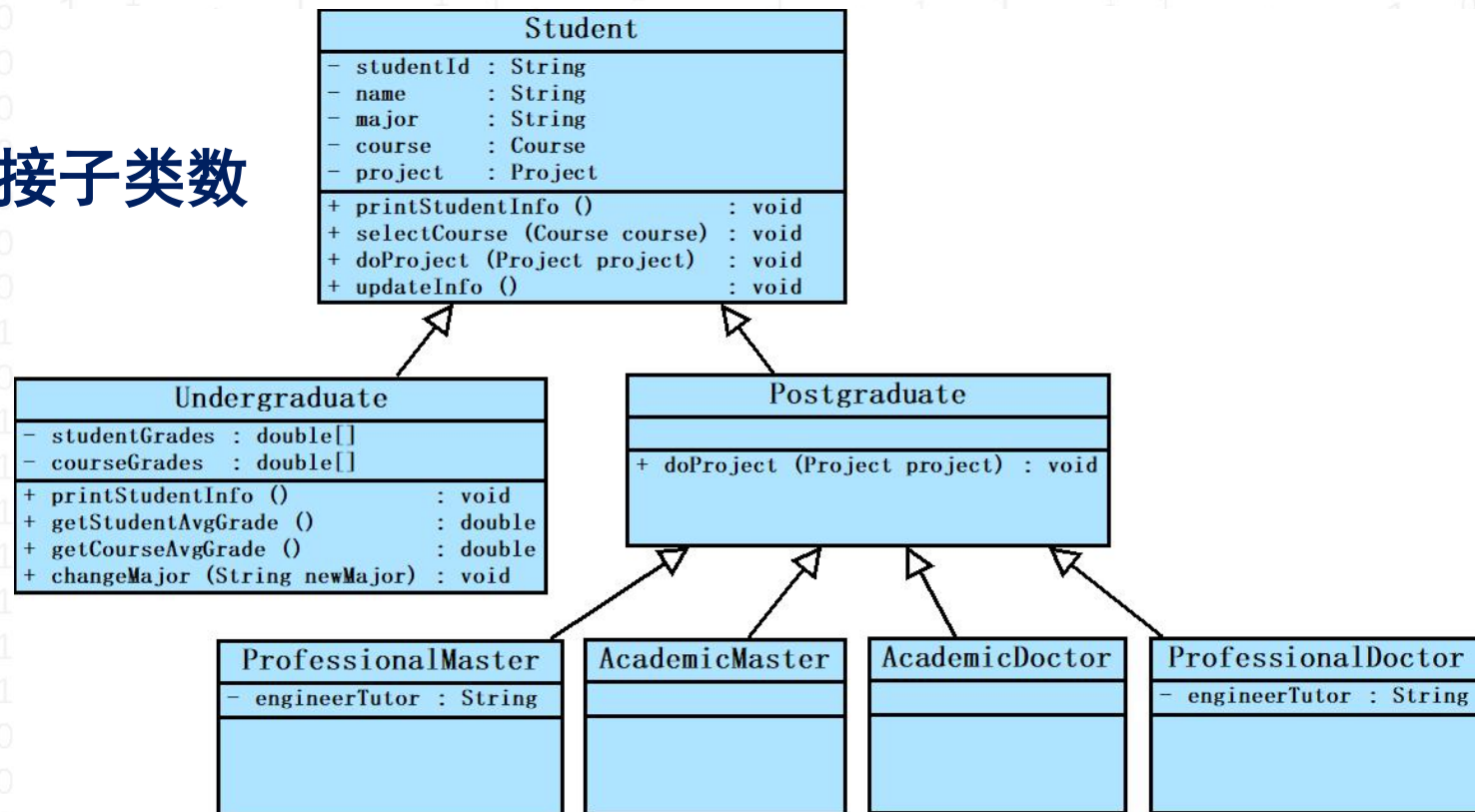
■ 度量实例

$\text{NOC}(\text{Student}) = 2$

$\text{NOC}(\text{Undergraduate}) = 0$

$\text{NOC}(\text{Postgraduate}) = 4$

$\text{NOC}(\text{AcademicDoctor}) = 0$



3. 面向对象度量——CK 度量方法

(4) 子类数 (Number Of Children, **NOC**)

■ 应用意义

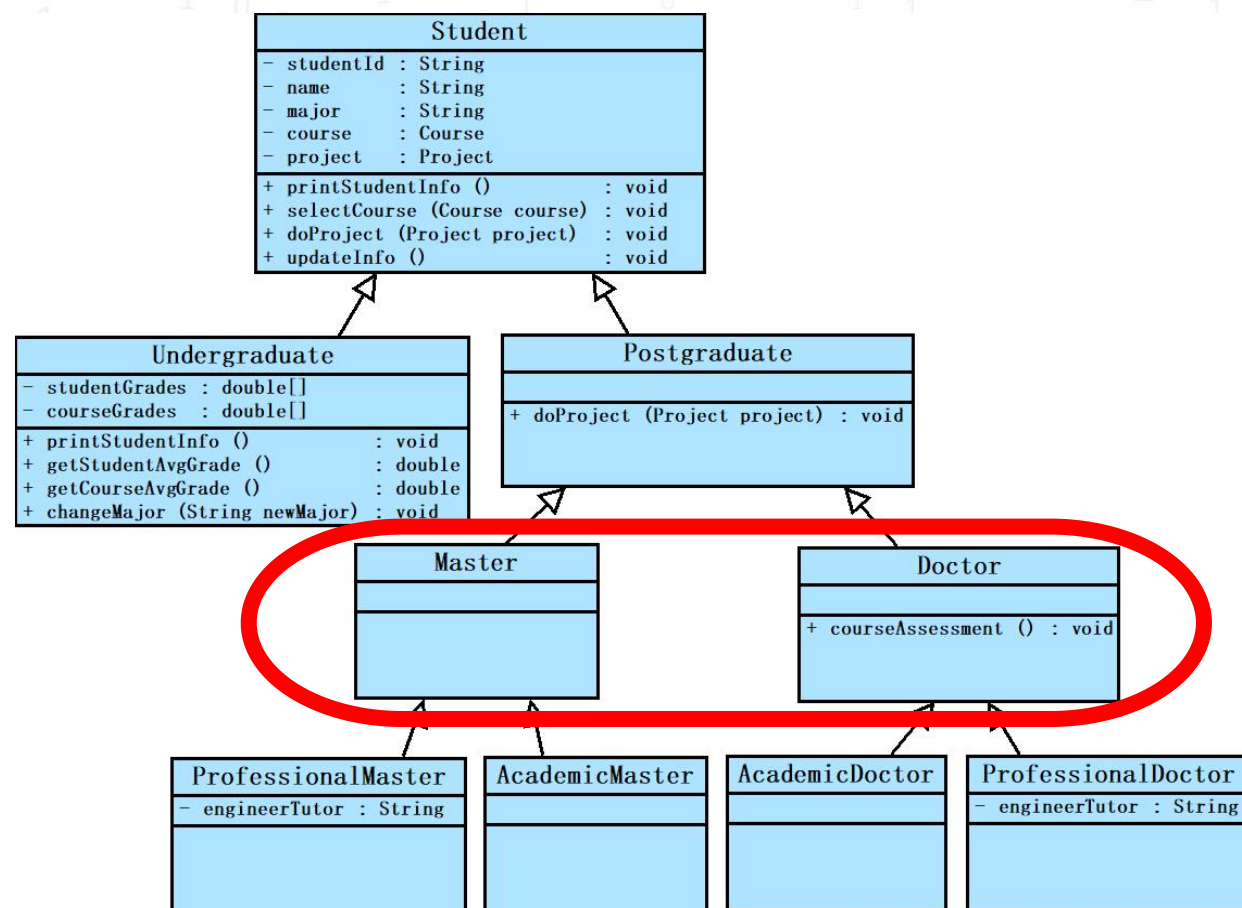
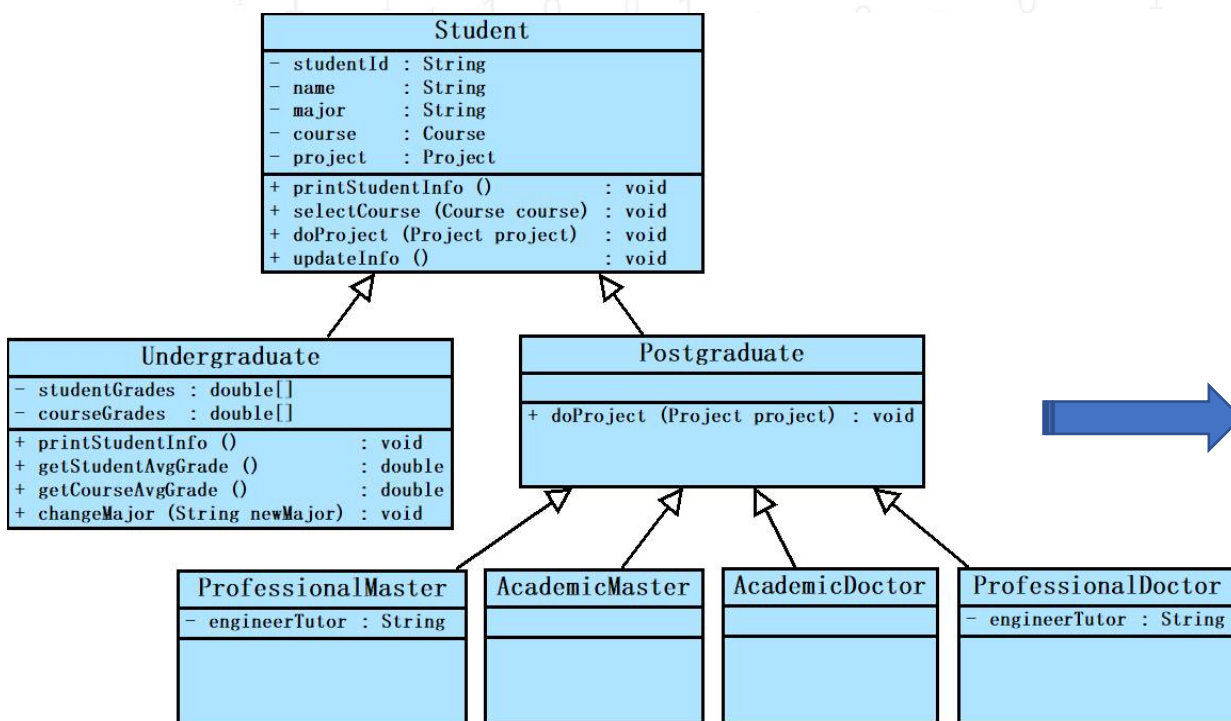
- 一个类的 NOC 反映类的重用性大小，也反映父类对子类的影响程度
- NOC 值越大，测试阶段需要对该类的方法进行更多测试
- NOC 值过大，则表明父类的抽象和继承的设计不够合理

如何改进设计？

3. 面向对象度量——CK 度量方法

(4) 子类数 (Number Of Children, **NOC**)

——改变继承层次结构，增加中间继承类



3. 面向对象度量——CK 度量方法

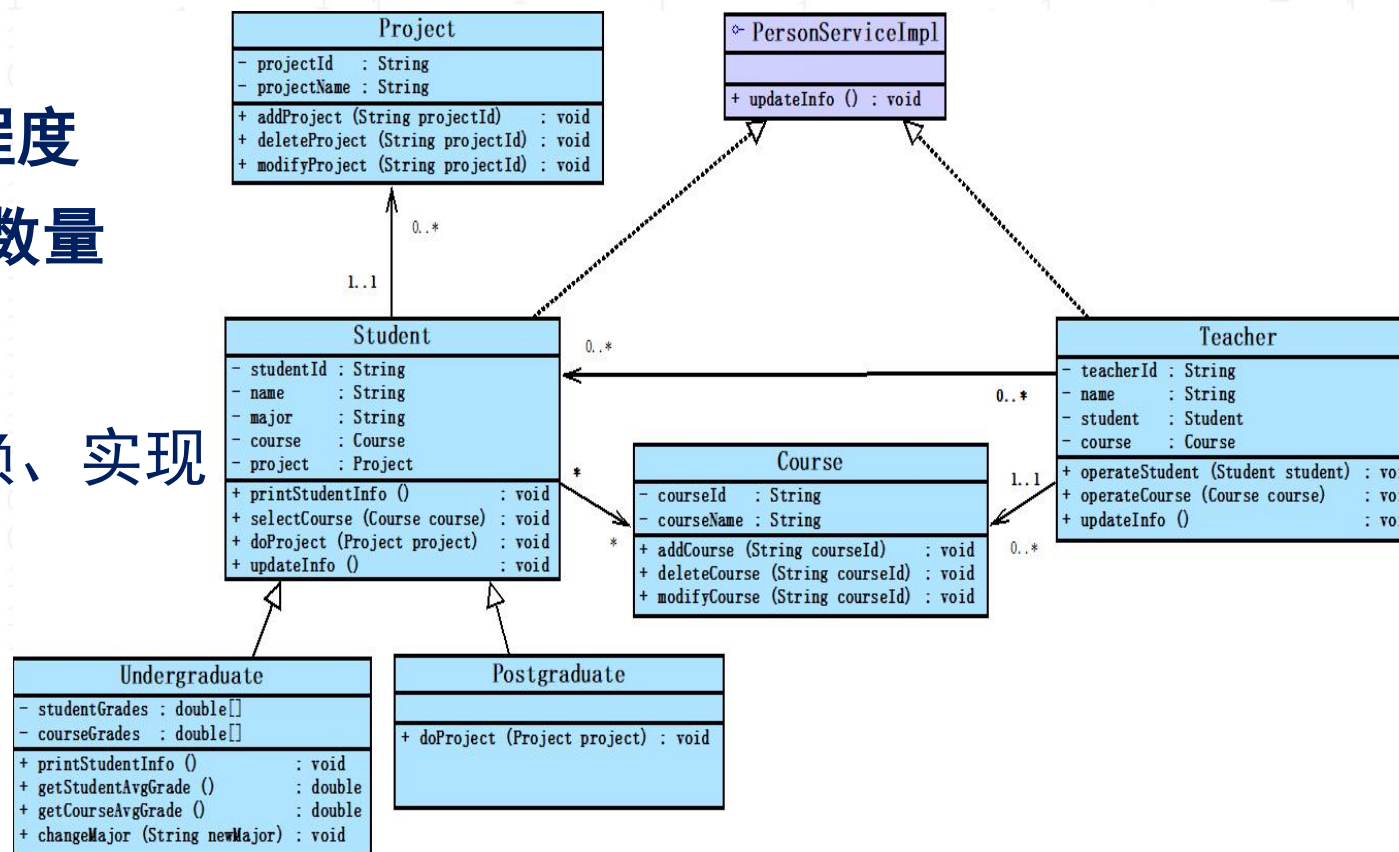
(5) 对象耦合 (Coupling Between Objects, CBO)

■ 度量方法

- 耦合是指模块之间联系的紧密程度
- 定义：CBO 指与该类耦合的类数量
- 类之间的关系
继承、关联、组合、聚合、依赖、实现
- 类之间的耦合关系
对象间的消息传递和调用

注意：

CBO 计算的耦合关系不包括继承关系



3. 面向对象度量——CK 度量方法

(5) 对象耦合 (Coupling Between Objects , **CBO**)

■ 度量实例

$CBO(\text{Student}) = 4$

$CBO(\text{Teacher}) = 3$

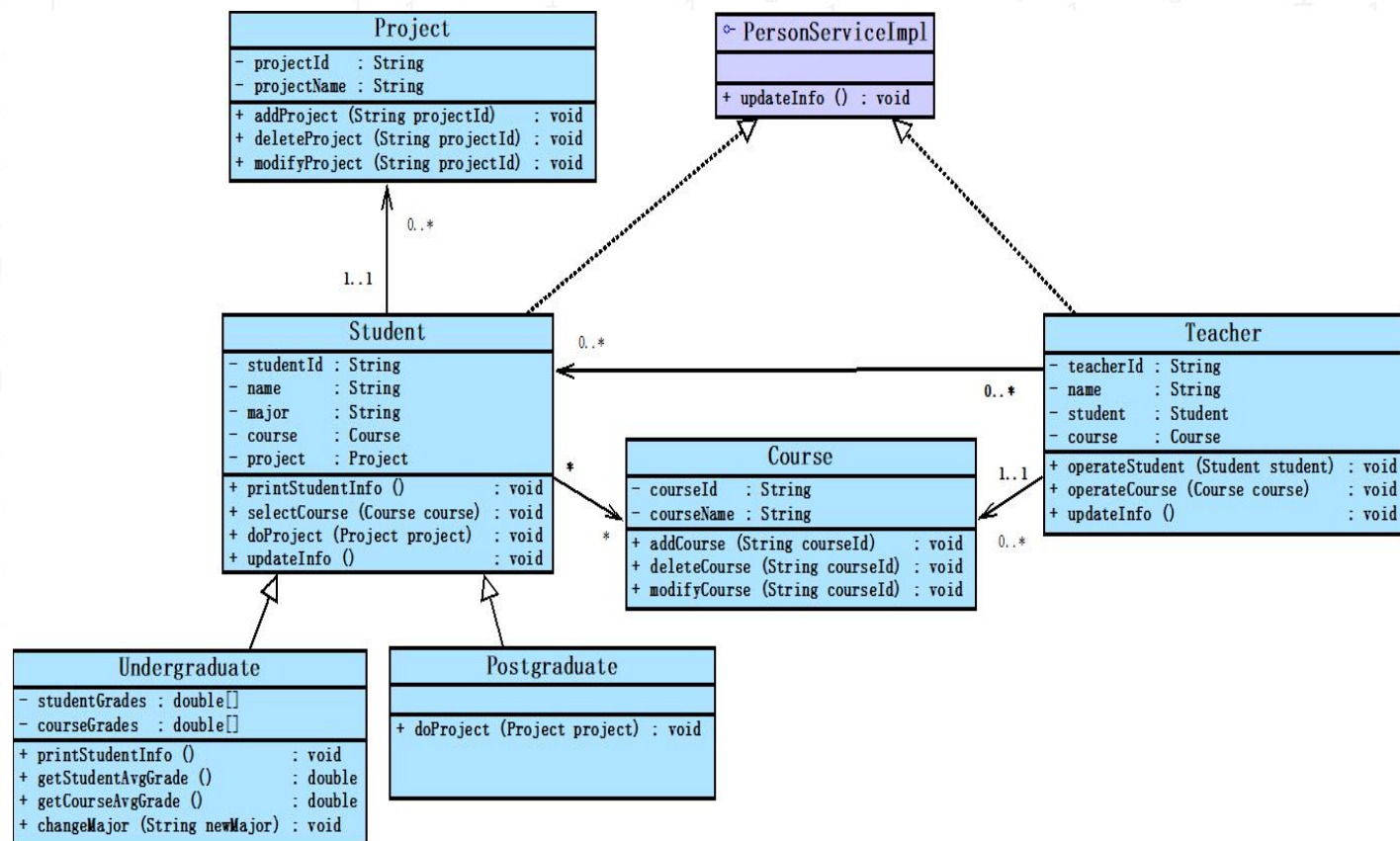
$CBO(\text{PersonServiceImpl}) = 2$

$CBO(\text{Course}) = 2$

$CBO(\text{Project}) = 1$

$CBO(\text{Undergraduate}) = 0$

$CBO(\text{Postgraduate}) = 0$



提问：上述七个类，哪个类更复杂？

3. 面向对象度量——CK 度量方法

(5) 对象耦合 (Coupling Between Objects , **CBO**)

■ 应用意义

- CBO 反映了类的模块化程度、类的封装性与重用性
- CBO 值越大，越容易受到其他类的设计变化的影响
- CBO 值过大，越不利于类的设计与重用，测试与维护也更难



代码坏味道：依恋情结 (Feature Envy)

如何改进设计？

——将大量使用其他类成员的方法迁移到被它所使用的那个类中

3. 面向对象度量——CK 度量方法

(6) 内聚缺乏度 (Lack of Cohesion in Methods, LCOM)

■ 理论基础

- **内聚：**是模块内部各个元素彼此结合的紧密程度
- **类内部元素**包括实例变量与类方法
- **类的内聚性**体现在类中的方法是相互关联或相似的、类中不同方法引用相同的实例变量
- 类的高内聚能够提升类的封装性

3. 面向对象度量——CK 度量方法

(6) 内聚缺乏度 (Lack of Cohesion in Methods, LCOM)

■ 度量方法

定义：类 C_k 中 n 个方法 $M_1, M_2, \dots, M_n$ ，
 n 个方法分别引用 n 个实例变量集合 $I_1, I_2, \dots, I_n$

$$\text{LCOM} = \begin{cases} |P| - |Q|, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$

$$P = \{(I_i, I_j) \mid (I_i \cap I_j) = \emptyset\}$$

$$Q = \{(I_i, I_j) \mid (I_i \cap I_j) \neq \emptyset\}$$

其中： I_i, I_j 是分别被方法 M_i, M_j 引用的实例变量集合

```
1  public class A {  
2      private string f1;  
3      private string f2;  
4      private int f3;  
5      private string f4;  
6  
7      public void method1() {  
8          //uses f1  
9      }  
10  
11     public void method2() {  
12         //uses f1  
13         //uses f2  
14     }  
15  
16     public void method3() {  
17         // I1 = {f3, f4}  
18         // uses f3  
19         // uses f4  
20     }  
21 }
```

3. 面向对象度量——CK 度量方法

(6) 内聚缺乏度 (Lack of Cohesion in Methods, LCOM)

■ 度量实例

方法对 (M_i, M_j)	$I_i \cap I_j$
(method1, method2)	$\{f_1\}$
(method1, method3)	$\emptyset$
(method2, method3)	$\emptyset$

$$LCOM(A) = 2 - 1 = 1$$

```
1 public class A {  
2     private string f1;  
3     private string f2;  
4     private int f3;  
5     private string f4;  
6  
7     public void method1() {  
8         //uses f1  
9     }  
10  
11     public void method2() {  
12         //uses f1  
13         //uses f2  
14     }  
15  
16     public void method3() {  
17         // I1 = {f3,f4}  
18         // uses f3  
19         // uses f4  
20     }  
21 }
```

3. 面向对象度量——CK 度量方法

(6) 内聚缺乏度 (Lack of Cohesion in Methods, LCOM)

■ 应用意义

- LCOM值越大表示类越缺乏内聚，类的实现更复杂，在类的程序实现阶段就更容易出错
- 类越缺乏内聚，意味着类的封装不合理

如何改进设计？

——将类分解成两个或更多的子类

3. 面向对象度量——CK 度量方法

(6) 内聚缺乏度 (Lack of Cohesion in Methods, LCOM)

■ 应用意义

```
1 public class A {  
2     private string f1;  
3     private string f2;  
4     private int f3;  
5     private string f4;  
6  
7     public void method1() {  
8         //uses f1  
9     }  
10  
11     public void method2() {  
12         //uses f1  
13         //uses f2  
14     }  
15  
16     public void method3() {  
17         // I1 = {f3,f4}  
18         // uses f3  
19         // uses f4  
20     }  
21 }
```

```
1 public class A {  
2     private string f1;  
3     private string f2;  
4  
5     public void method1() {  
6         //uses f1  
7     }  
8  
9     public void method2() {  
10        //uses f1  
11        //uses f2  
12    }  
13  
14 }
```

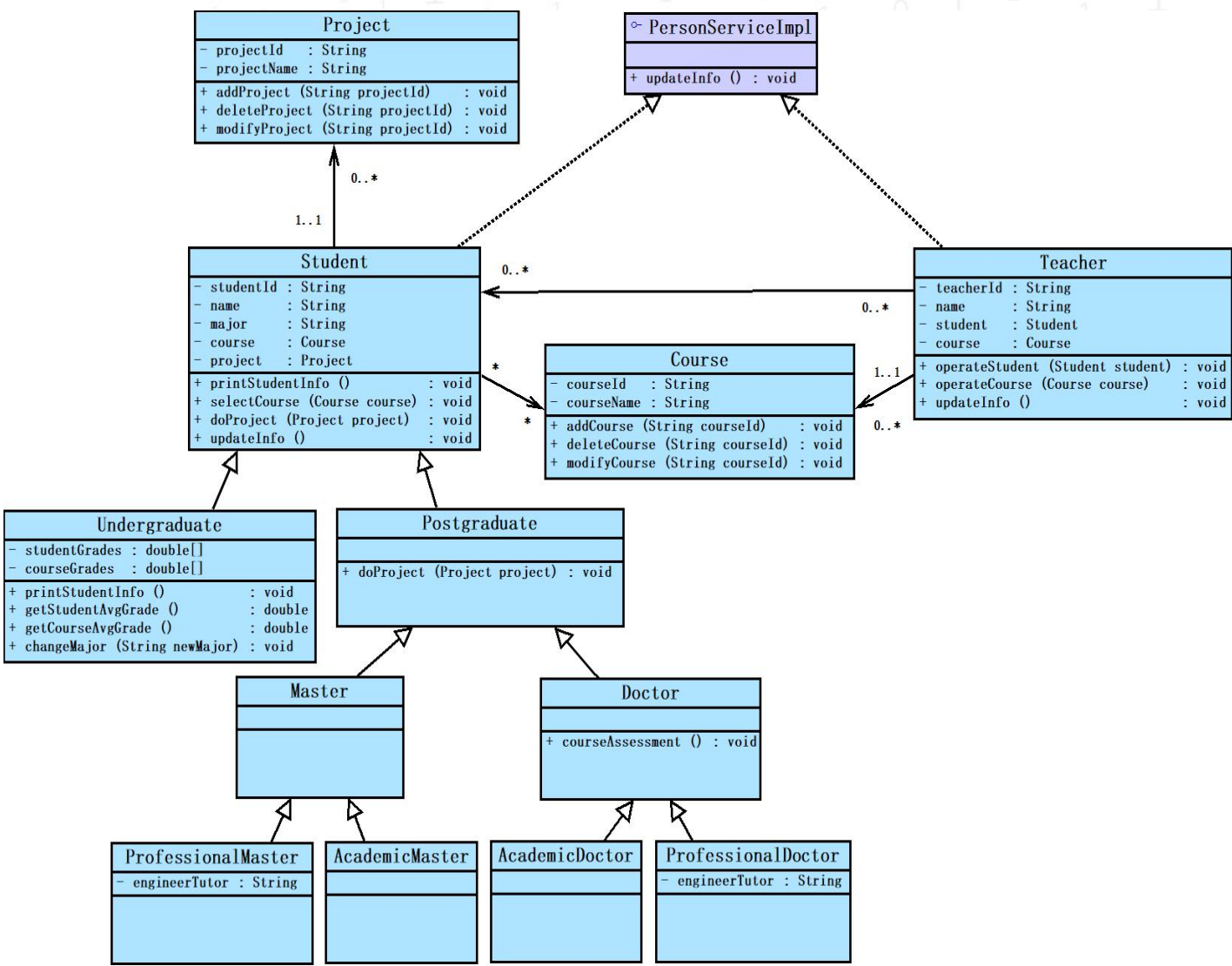
```
1 public class B {  
2     private int f3;  
3     private string f4;  
4  
5     public void method3() {  
6         // I1 = {f3,f4}  
7         // uses f3  
8         // uses f4  
9     }  
10 }
```

4. CK 度量方法应用与实现

基于CK度量能够建构度量算法并实现自动化度量

4. CK 度量方法的应用与实现

■ 应用实战：利用CK 度量方法分析“教务管理系统”的类图与代码



A组同学度量:

- WMC(Student)
- RFC(Student)
- DIT(Student)
- NOC(Student)
- CBO(Student)
- LCOM(Student)

B组同学度量:

- WMC(Undergraduate)
- RFC(Undergraduate)
- DIT(Undergraduate)
- NOC(Undergraduate)
- CBO(Undergraduate)
- LCOM(Undergraduate)

4. CK 度量方法的应用与实现

■ 应用实战：Student类代码

```
public class Student {  
     String studentId; //学生Id  
    String name; //学生姓名  
    String major; //学生专业  
    Course course; //选择的课程  
    Project project; //参与的项目  
  
    //打印学生基本信息  
    public void printStudentInfo() {  
        //use {studentId, name, major}  
    }  
    //选课  
    public void selectCourse(Course course) {  
        //use {studentId, course}  
    }  
    //做项目  
    public void doProject(Project project) {  
        //use {project}  
        project.modifyProject(project.getProjectId());  
    }  
    //更新信息  
    public void updateInfo() {  
        //use {studentId, name, major, course, project}  
    }  
}
```

Undergraduate类代码

```
public class Undergraduate extends Student {  
     //继承 studentId, name, major  
    double[] studentGrades; //学生分数  
    double[] courseGrades; //课程分数  
  
    //打印学生基本信息  
    public void printStudentInfo() {  
        //use {studentId, name, major}  
    }  
  
    //计算学生平均成绩  
    public double getStudentAvgGrade() {  
        //use {studentGrades}  
    }  
  
    //计算课程平均分  
    public double getCourseAvgGrade() {  
        //use {courseGrades}  
    }  
  
    //转专业  
    public void changeMajor(String newMajor) {  
        //use {studentId, major}  
    }  
}
```


4. CK 度量方法的应用与实现

■ 应用实战

WMC(Student) = 4

RFC(Student) = 6

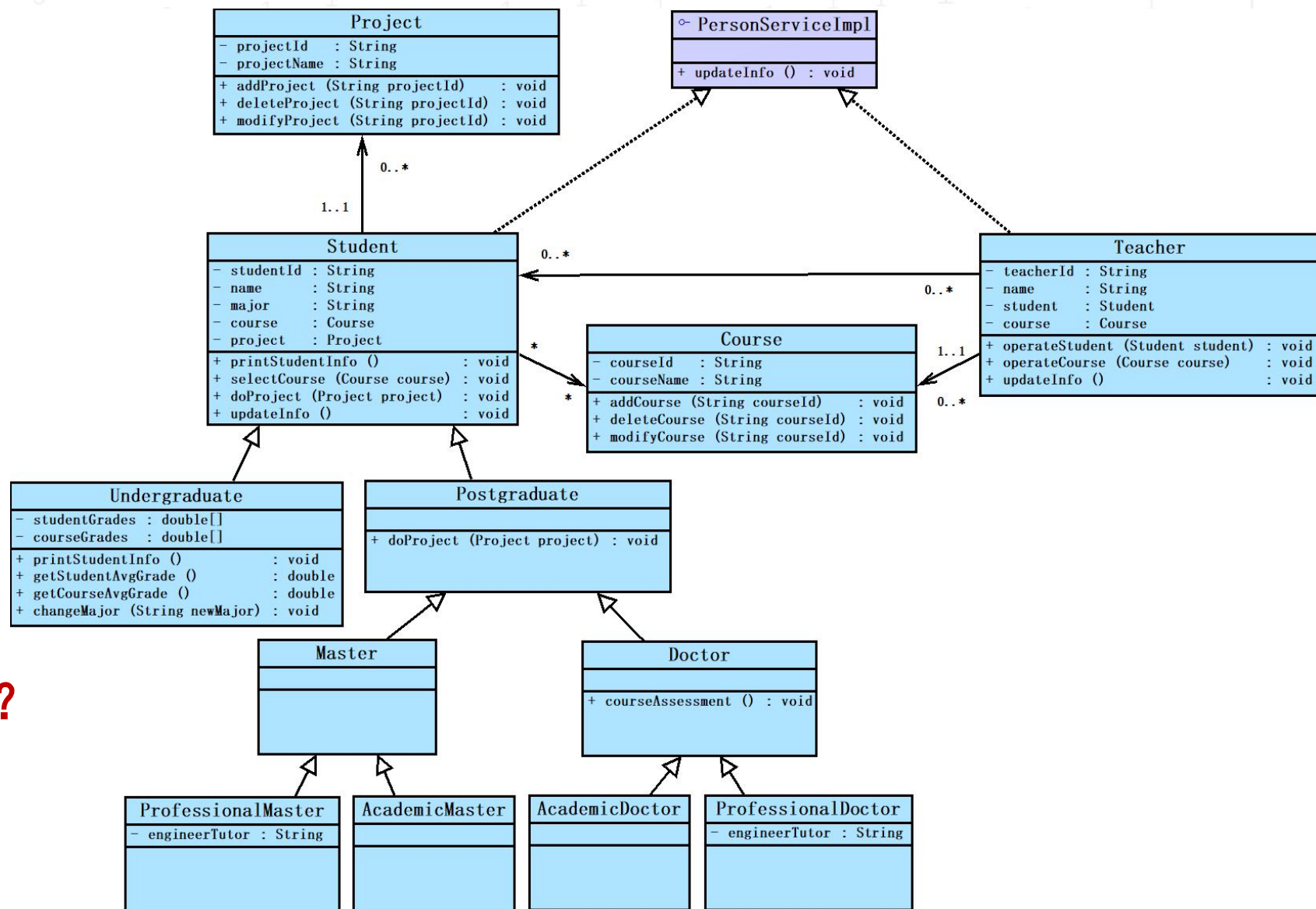
DIT(Student) = 0

NOC(Student) = 2

CBO(Student) = 4

LCOM(Student) = 4

如何改进设计与实现？



4. CK 度量方法的应用与实现

■ 应用实战

WMC(Student) = 4

RFC(Student) = 6

DIT(Student) = 0

NOC(Student) = 2

CBO(Student) = 4

LCOM(Student) = 0

发现问题:

- RFC值与CBO值有关联?
- 类中方法越多, 方法对越多, 计算得到的LCOM相对更大?

```
1 public class Student {
2     String studentId; //学生Id
3     String name;      //学生姓名
4     String major;     //学生专业
5     Course course;    //选择的课程
6     Project project;  //参与的项目
7
8     //打印学生基本信息
9     public void printStudentInfo() {
10         //use {studentId, name, major}
11     }
12     //选课
13     public void selectCourse(Course course) {
14         //use {studentId, course}
15     }
16     //做项目
17     public void doProject(Project project) {
18         //use {project}
19         project.modifyProject(project.getProjectId());
20     }
21     //更新信息
22     public void updateInfo() {
23         //use {studentId, name, major, course, project}
24     }
25 }
```

4. CK 度量方法的应用与实现

■ 问题讨论：如何优化LOCM 度量方法？

定义：类 C_k 中 n 个方法 $M_1, M_2, \dots, M_n$,

n 个方法分别引用 n 个实例变量集合 $I_1, I_2, \dots, I_n$

$$\text{LOCM} = \begin{cases} |P| - |Q|, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$



$$\text{LOCM} = \begin{cases} \frac{|P| - |Q|}{n}, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$

$$P = \{(I_i, I_j) \mid (I_i \cap I_j) = \emptyset\}$$

$$Q = \{(I_i, I_j) \mid (I_i \cap I_j) \neq \emptyset\}$$

其中： I_i, I_j 是分别被方法 M_i, M_j 引用的实例变量集合

4. CK 度量方法的应用与实现

■ 如何优化LCOM 度量方法？

定义：类 C_k 中 n 个方法 $M_1, M_2, \dots, M_n$,

n 个方法分别引用 n 个实例变量集合 $I_1, I_2, \dots, I_n$

$$\text{LOCM} = \begin{cases} |P| - |Q|, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$



$$\text{LOCM} = \begin{cases} \frac{|P| - |Q|}{n}, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$

$$P = \{(I_i, I_j) \mid (I_i \cap I_j) = \emptyset\}$$

$$Q = \{(I_i, I_j) \mid (I_i \cap I_j) \neq \emptyset\}$$

其中： I_i, I_j 是分别被方法 M_i, M_j 引用的实例变量集合

4. CK 度量方法的应用与实现

■ 如何优化LOCM 度量方法？

定义：类 C_k 中 n 个方法 $M_1, M_2, \dots, M_n$,

n 个方法分别引用 n 个实例变量集合 $I_1, I_2, \dots, I_n$

$$\text{LOCM} = \begin{cases} |P| - |Q|, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$



$$\text{LOCM} = \begin{cases} \frac{|P| - |Q|}{n}, & |P| > |Q| \\ 0, & |P| \leq |Q| \end{cases}$$

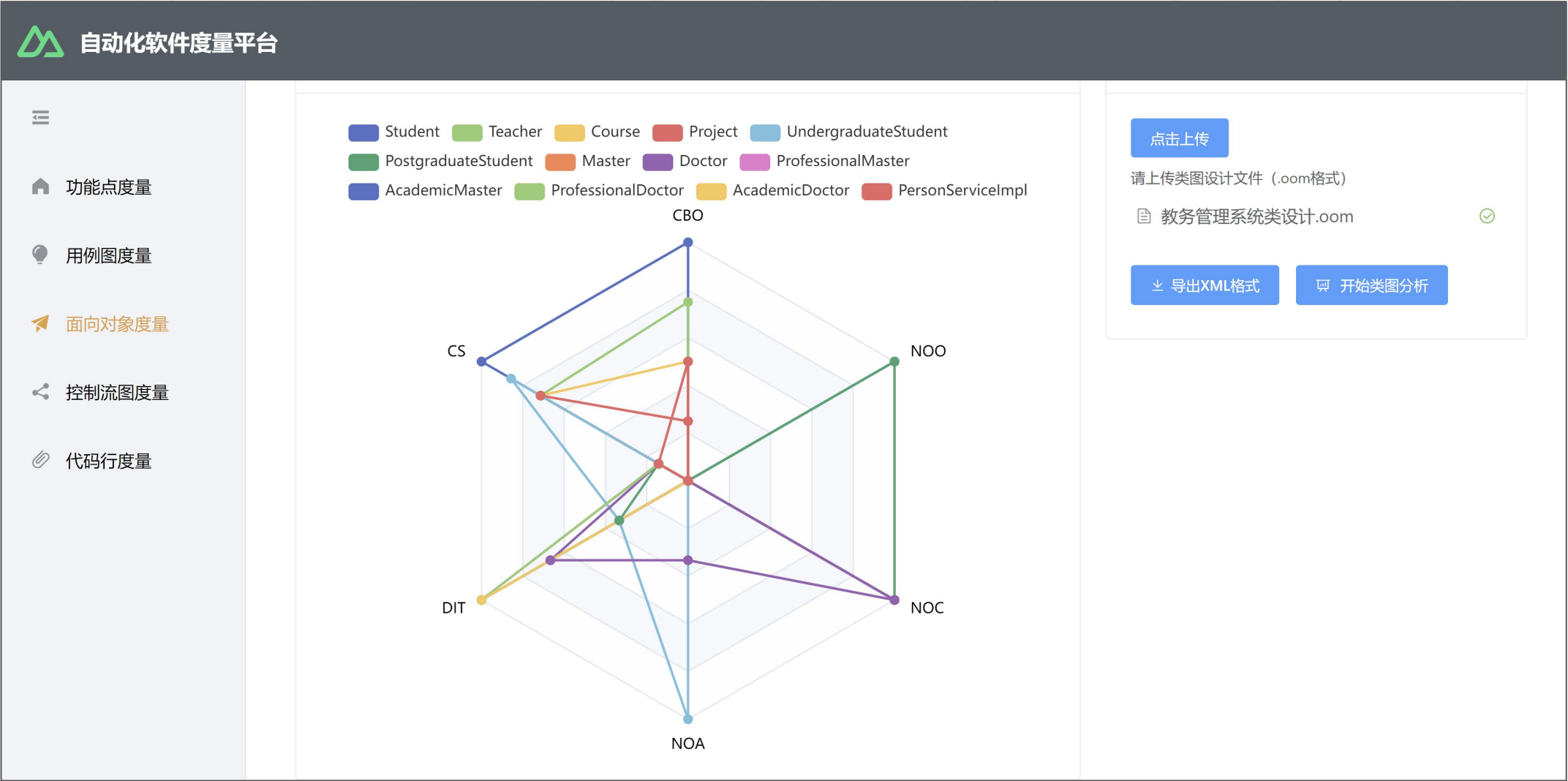
$$P = \{(I_i, I_j) \mid (I_i \cap I_j) = \emptyset\}$$

$$Q = \{(I_i, I_j) \mid (I_i \cap I_j) \neq \emptyset\}$$

其中： I_i, I_j 是分别被方法 M_i, M_j 引用的实例变量集合

4. CK 度量方法的应用与实现

■ 2020级综合实验“自动化软件度量平台”展示



4. CK 度量方法的应用与实现

■ 2020级综合实验“自动化软件度量平台”展示

自动化软件度量平台							
☰ 🏠 功能点度量 💡 用例图度量 🚩 面向对象度量 🔗 控制流图度量 📎 代码行度量	类名	对象间的耦合度 CBO	类规模度量 CS	方法重写数 NOO	增加方法数目 NOA	子类数目 NOC	继承树深度 DIT
	Student	4	7	0	0	2	0
	Teacher	3	5	0	0	0	0
	Course	2	5	0	0	0	0
	Project	1	5	0	0	0	0
	UndergraduateStudent	0	6	1	3	0	1
	PostgraduateStudent	0	1	1	0	2	1
	Master	0	0	0	0	2	2
	Doctor	0	1	0	1	2	2
	ProfessionalMaster	0	1	0	0	0	3

总结

■ 基于CK 度量的结果改进面向对象设计与实现，提升面向对象软件质量

度量点	度量对象	度量与改进阶段
类加权方法数 (Weighted Methods per Class , WMC)	类图	软件设计阶段
继承树深度 (Depth of Inheritance Tree , DIT)		
孩子数 (Number of Child, NOC)		
对象耦合 (Coupling Between Objects , CBO)		
类响应 (Response For a Class , RFC)	类代码	软件实现阶段
内聚缺乏度(Lack of COhesion in Methods , LCOM)		